

# 편도훈

## AI 서비스를 만드는 백엔드 엔지니어

Java-Spring으로 서비스의 기반을 만들고, AI를 실제 업무에 연결합니다. 기획부터 개발, 배포와 운영까지. 끝까지 작동하는 시스템을 만듭니다.

dohoon321@gmail.com

<https://github.com/bigbulgogiburger>

### 핵심 역량

**Backend** — Java 21 · Spring Boot · JPA · QueryDSL · Spring Security · MSA · Event-driven

**Applied AI** — LLM API Integration · Tool Calling · HITL · Agent Workflow · Quality Gates

**Operations** — AWS · Docker · Jenkins · Prometheus · Grafana · Redis · MySQL · Spring Batch

### 경력

#### DB Inc. · PM · Backend Engineer

2023.11 — 현재

A/S 플랫폼의 기획·개발·운영을 맡고 있습니다. CS AI Agent 도입, 백엔드 전환, 배포 자동화와 신규 업무 시스템 구축을 수행했습니다.

#### 교육지대 · Backend Engineer

2023.01 — 2023.11

태블릿 과외 서비스의 통합회원 연동과 모니터링 도입. 실제 서비스가 안정적으로 운영되도록 핵심 기능을 개선했습니다.

#### 플랫폼비 · Backend Engineer

2021.11 — 2023.01

MSA 결제 서버와 Apple 인앱 결제 연동을 개발했습니다. 결제·환불의 정합성과 서비스 간 인증을 다뤘습니다.

#### 링커 · Backend Engineer

2021.05 — 2021.11

통합회원 이벤트, Redis 기반 Q&A, Apple 탈퇴 연동과 Spring Batch를 개발했습니다.

# AI를 실제 업무에 연결하다

## A/S 플랫폼 · CS AI Agent

반복되는 문의를 이해하고, 허용된 업무만 수행하는 AI Agent. 모델의 답변을 실제 서비스의 동작으로 연결했습니다.

PM · Backend · AI Integration · DevOps

Java 21 · Spring Boot · LLM API · Tool Calling · HITL

### 문제

일반적인 LLM은 접수 상태나 고객의 업무 맥락을 알지 못합니다. 운영 데이터를 조회하면서도, 잘못된 답변이 실제 업무 변경으로 이어지지 않도록 통제해야 했습니다.

### 설계와 구현

#### 답변과 실행의 경계를 설계

상용-로컬 LLM을 API로 연결하고, 화이트리스트에 등록된 도구만 호출하도록 구성했습니다. 업무 권한은 백엔드에서 검증합니다.

#### 중요한 변경은 사람이 결정

접수 상태 변경과 같은 동작에 HITL 승인 절차를 적용했습니다. 조회와 변경을 구분해 AI가 할 수 있는 일을 명확하게 제한했습니다.

#### 운영 플랫폼까지 함께 개선

Next.js 백엔드 API 약 50개를 Spring으로 이관하고 MongoDB를 MySQL로 전환했습니다. GitLab·Jenkins·Docker 배포와 모니터링을 함께 구축했습니다.

### 성과

운영 업무에 CS AI Agent를 도입했습니다. 엑셀 업로드는 100건 3분 이상에서 500건 수초로 개선했고, 수동 약 1시간이던 배포를 자동 약 10분으로 단축했습니다. AI 응대 자동화율은 별도 측정치로 제시하지 않습니다.

좋은 AI 서비스는 답변을 생성하는 능력만큼, 무엇을 실행하지 않을지 결정하는 구조가 중요합니다.

# AI 개발에도 품질의 기준을

## claude\_jira\_harness · MIT Open Source

요구사항 한 줄에서 구현과 검증까지. AI의 작업을 연결하고, 리뷰를 통과하지 못한 변경은 commit 단계에서 차단합니다.

설계 · 개발 · 실제 프로덕트 적용

Agent Workflow · Jira · Git Hooks · Quality Gate

### 문제

AI로 빠르게 코드를 생성해도 요구사항의 맥락과 검증 기준이 이어지지 않으면 재작업이 늘어납니다. 생성과 검증을 하나의 반복 가능한 개발 절차로 묶어야 했습니다.

### 설계와 구현

#### 요구사항을 실행 가능한 흐름으로

create → start → clarify → plan → execute → test → commit → complete의 8단계로 작업을 연결했습니다. 앞 단계의 산출물이 다음 단계의 입력이 됩니다.

#### 검증을 권고가 아닌 게이트로

전문 관점의 에이전트가 fan-out 리뷰를 수행합니다. PASS를 받지 못한 변경은 Git Hook의 commit 게이트에서 차단하도록 구성했습니다.

#### 작업 결과를 지식으로 축적

개발 산출물을 위키에 연결하고 정합성을 점검합니다. 중단 후 재개할 때에도 작업 맥락과 근거를 이어갈 수 있도록 설계했습니다.

### 성과

업무용 A/S 플랫폼, CS, 홈페이지, App 등 4개 프로덕트에 적용했습니다. 슬래시 명령과 스킴으로 구성된 프레임워크를 MIT 라이선스로 공개했습니다.

AI에게 일을 맡기는 것에서 한 걸음 더. 결과를 받아들이기 기준까지 코드로 만듭니다.

[https://github.com/bigbulgogiburger/claude\\_jira\\_harness](https://github.com/bigbulgogiburger/claude_jira_harness)

# 복잡한 업무를 하나의 시스템으로

## 글로벌 전동공구사·A/S 관리 플랫폼

접수부터 정산까지, 흩어진 업무 규칙을 도메인과 상태로 정리했습니다. AI 개발 워크플로를 활용해 기획·백엔드·화면·검증을 단독으로 연결했습니다.

도메인 설계 · Full-stack · QA · DevOps

Spring Boot · Vue 3 · MySQL · RBAC · Flyway

### 문제

와이어프레임 중심의 기획에는 역할, 권한, 상태 전이와 도메인 경계가 충분히 정의되어 있지 않았습니다. 개발에 앞서 서로 다른 업무 용어와 규칙을 일관된 모델로 정리해야 했습니다.

### 설계와 구현

#### 코드와 기획을 함께 확인

요구사항과 데이터 구조를 대조해 도메인 경계를 구체화했습니다. 고객사·거래처·역할별 책임을 분리하고 기준 문서에 연결했습니다.

#### 권한과 상태를 시스템의 중심에

역할 기반 접근 제어와 업무 상태 전이를 설계했습니다. 화면별 기능 구현이 전체 업무 흐름과 어긋나지 않도록 기준을 맞췄습니다.

#### 혼자 개발해도 검증은 여러 관점으로

직접 만든 harness를 활용해 작업을 분리하고 전문 관점의 리뷰를 적용했습니다. 변경 수용 여부를 품질 게이트로 통제했습니다.

### 성과

접수·수리·정산의 업무 흐름을 다루는 A/S 관리 플랫폼을 단독 구축했습니다. 도메인·권한·상태 전이를 일관된 기준으로 연결하고, AI 개발 워크플로와 품질 게이트를 실제 업무 시스템 개발에 적용했습니다.

구현 속도는 문제의 경계를 정확하게 정의하는 데서 시작합니다.

# 결제는 끝까지 정확해야 하니까

## 1:1 태블릿 과외 · 결제 마이크로서비스

독립된 결제 서버부터 Apple 환불 동기화, 장애 관측까지. 돈과 연결된 상태가 일관되게 유지되도록 설계했습니다.

Backend · Payment Integration · Observability

Java · Spring · MSA · Apple IAP · Prometheus

### 문제

신규 결제 기능을 기존 서버와 분리하면서 내부 인증과 외부 결제 연동을 함께 해결해야 했습니다. Apple에서 발생하는 취소·환불 이벤트도 서비스 DB에 반영해야 했습니다.

### 설계와 구현

#### 결제를 독립된 책임으로 분리

MSA 결제 서버를 구축하고 Internal-External 통신과 인증 흐름을 분리했습니다. 기존 서비스와 독립적으로 결제 기능을 개발할 수 있게 했습니다.

#### 외부 이벤트를 내부 상태와 동기화

Apple Webhook-Verify API와 재시도를 연결했습니다. Spring AOP의 self-invocation 문제를 해결해 재시도가 의도대로 동작하도록 보완했습니다.

#### 운영 중인 상태를 볼 수 있게

Prometheus·Grafana·Pinpoint로 지표와 트레이스를 확인하도록 구성했습니다. 사용자의 문의 이후에 알게 되던 장애를 사전에 탐지하는 체계로 바꿨습니다.

### 성과

독립 결제 서버 구축으로 결제 출시 일정을 약 1개월 단축했습니다. Apple 환불 상태 동기화와 관측 도구를 통해 결제 운영의 정합성과 가시성을 높였습니다.

정상 요청 하나의 성공보다, 실패 이후에도 상태를 복구할 수 있는 구조를 고민합니다.

# 서비스를 잇고, 결합은 낮추고

## 입시 정보 플랫폼 · 통합회원과 Q&A

AWS SNS 이벤트로 회원 정보를 연결하고, Redis로 별도 가입 없는 Q&A 경험을 만들었습니다.

Backend · Event Architecture · Batch

AWS SNS · Redis · Spring Batch · MySQL

### 문제

서로 다른 서비스의 회원을 연결해야 했지만 DB를 직접 공유하기는 어려웠습니다. 기존 스키마 변경을 최소화하면서 신규 가입 없이 Q&A를 이용할 수 있어야 했습니다.

### 설계와 구현

#### DB 공유 대신 이벤트로 연결

AWS SNS Pub/Sub를 이용해 회원 변경을 전달했습니다. 서비스 사이의 직접적인 결합을 낮추면서 통합회원 전환을 진행했습니다.

#### Redis로 임시 회원 맥락 유지

외부 서비스의 회원 ID와 상담 정보를 매핑해 임시 회원 세션을 구성했습니다. DB 스키마 변경 없이 Q&A와 멘토 매칭을 연결했습니다.

#### 흩어진 운영 작업 통합

분산된 스케줄러 22개를 Spring Batch 서버 1대로 통합했습니다. Apple 탈퇴 revoke 연동과 토큰 갱신 작업도 처리했습니다.

### 성과

전체 회원 약 10만 중 통합 대상 약 1만 명을 무에러 전환했습니다. Q&A는 약 1,000건을 처리했고, DB 스키마 변경 없이 서비스 간 회원 경험을 연결했습니다.

서비스가 서로 협력하되, 각자의 변경이 다른 서비스의 부담이 되지 않도록 설계합니다.